

Introdução à Computação

arrays e strings

Alexandre Rademaker

Array I

scores0.c

```
1 #include <stdio.h>
2
3 int main(void) {
4
5     int score1 = 72;
6     int score2 = 73;
7     int score3 = 33;
8
9     printf("Average: %d\n",
10         (score1 + score2 + score3) / 3.0);
11 }
```

Nomes distintos para cada posição na memória! Não necessariamente alocados de forma contínua! Podemos simplificar?

Array II

Isto torna o código mais elegante?

scores1.c

```
1 #include <stdio.h>
2
3 int main(void) {
4
5     int scores[] = {1, 2, 3};
6
7     printf("Average: %f\n", (scores[1] + scores[2] + scores[3]) / 3.0);
8 }
```

Array III

endereço	valor	
0x10000	?	
0x10004	72	← score1
0x10008	?	
0x10012	?	
0x10016	73	← score2
0x10020	?	
0x10024	?	
0x10028	33	← score3
0x10032	?	
0x10036	?	
0x10040	?	

endereço	valor	
0x10000	?	
0x10004	72	← scores
0x10008	73	
0x10012	33	
0x10016	?	
0x10020	?	
0x10024	?	
0x10028	?	
0x10032	?	
0x10036	?	
0x10040	?	

caracteres e inteiros I

hi0.c

```
1 #include <stdio.h>
2
3 int main(void) {
4
5     char c1 = 'H';
6     char c2 = 'I';
7     char c3 = '!';
8
9     printf("%c%c%c\n", c1, c2, c3);
10    printf("%i %i %i\n", c1, c2, c3);
11 }
```

```
1 > clang hi0.c
2 > ./a.out
3 HI!
4 72 73 33
```

caracteres e inteiros II

Os `char` são armazenados como inteiros (códigos ASCII). `printf` fez uma conversão **implícita** usando `%i`; também podemos converter explicitamente:

```
1 printf("%i %i %i\n", (int) c1, (int) c2, (int) c3);
```

Mas conversões podem **perder** informação, por exemplo, de `float` para `int`.

Strings I

Representamos **strings** como arrays de caracteres (cada um com 8 bits ou um byte), um tipo que na verdade não é primitivo em C.

hi1.c

```
1 #include <stdio.h>
2
3 int main(void) {
4
5     char a[] = "HI!";
6     char b[] = {'H', 'I', '!', '\0'};
7
8     printf("%s vs %s\n", a, b);
9
10    // acesso a cada char por indice -- note o '\0'
11    // final em a[3]
12    for (int i = 0; i < 4; i++) {
13        printf("a[%d] = %c (%i)\n", i, a[i], a[i]);
14    }
```

Strings II

```
1 > ./a.out
2 HI! vs HI!
3 a[0] = H (72)
4 a[1] = I (73)
5 a[2] = ! (33)
6 a[3] = (0)
```

H	I	!	
c1	c2	c3	

H	I	!	\0	
s[0]	s[1]	s[2]	s[3]	

Em C, strings terminam com o char `'\0'` ou inteiro `0` (um byte com todos os bits 0), marcando a **terminação** da string. Este é o chamado 'NUL'. Sempre precisamos de um byte adicional.

Strings III

Outros tipos de dados de tamanho **fixo** não precisam de tal mecanismo. Mas bibliotecas que oferecem representação de variadas precisões numéricas podem usar também tal recurso.

Strings IV

hi2.c

```
1 #include <stdio.h>
2
3 int main(void) {
4
5     char s[] = "HI!";
6     char t[] = {'B', 'y', 'e', '!'};
7
8     // qual e o problema com t? nao tem '\0' terminador
9     for (int i = 0; i < 4; i++)
10         printf("loc s[%d] = %c %p\n", i, s[i], &s[i]);
11     for (int i = 0; i < 4; i++)
12         printf("loc t[%d] = %c %p\n", i, t[i], &t[i]);
13
14     printf("%s and %s\n", t, s);
15 }
```

Strings V

s[0]	H
s[1]	i
s[2]	!
s[3]	\0

t[0]	B
t[1]	y
t[2]	e
t[3]	!

Strings VI

Um array de strings ou um array de array de caracteres.

hi3.c

```
1 #include <stdio.h>
2
3 int main(void) {
4
5     char arr[3][10] = {"Geek", "Geeks", "Geekfor"};
6
7     for (int i = 0; i < 3; i++) {
8         printf("%s\n", arr[i]);
9     }
10 }
```

Strings VII

Na memória, este array está armazenado da seguinte forma.

	0	1	2	3	4	5	6	7	8	9
arr [0]	G	e	e	k	\0					
arr [1]	G	e	e	k	s	\0				
arr [2]	G	e	e	k	s	f	o	r	\0	

Memory Wastage